



CLASE DE DOMINIO CON SPRING



spring



spring
boot

Guía Paso a Paso – Creación de la Clase de Dominio `Persona` con Spring Boot y Thymeleaf

Introducción

En esta lección vamos a aprender cómo crear una **clase de dominio llamada `Persona`** en un proyecto Spring Boot con soporte para Thymeleaf. Esta clase nos servirá para **compartir información dinámica** con la vista `index.html`, aplicando el patrón MVC. 💡

Paso 1: Crear la clase `Persona.java`

Dentro del proyecto `HolaMundoThymeleaf`, ve a la siguiente carpeta:

src/main/java/gm/domain

Da clic derecho en el paquete `domain` y selecciona **New > Java Class**.

Nombre de la clase: `Persona`

Asegúrate que esté en el paquete: `gm.domain`

 Nota importante: Ya no utilizaremos las anotaciones de lombok ya que han causado problemas al actualizar a nuevas versiones.

 El contenido debe ser el siguiente:

```
package gm.domain;

public class Persona {
    private String nombre;
    private String apellido;
    private String email;
    private String telefono;

    public Persona() {
    }

    public Persona(String nombre, String apellido, String email, String telefono) {
        this.nombre = nombre;
        this.apellido = apellido;
        this.email = email;
        this.telefono = telefono;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getApellido() {
        return apellido;
    }

    public void setApellido(String apellido) {
        this.apellido = apellido;
    }
}
```

```
public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public String getTelefono() {
    return telefono;
}

public void setTelefono(String telefono) {
    this.telefono = telefono;
}

@Override
public String toString() {
    return "Persona{" + "nombre=" + nombre + ", apellido=" + apellido + ", email=" + email + ",
telefono=" + telefono + '}';
}
}
```

✚ Explicación:

- Esta es la clase de dominio del sistema que vamos a estar utilizando.
- Agregamos los constructores, métodos `get`, `set` y `toString`.

Paso 2: Modificar el archivo `ControladorInicio.java`

Ubicación:

`src/main/java/gm/ControladorInicio.java`

Modifica el contenido del método `inicio()` para crear un objeto de tipo `Persona` y compartirlo con la vista.

▼ Código modificado del controlador:

```
package gm;

import gm.domain.Persona;
import org.slf4j.Logger;
```

```
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class ControladorInicio {

    private static final Logger log = LoggerFactory.getLogger(ControladorInicio.class);

    @Value("${index.saludo}")
    private String saludo;

    @GetMapping("/")
    public String inicio(Model model){
        var mensaje = "Mensaje con Thymeleaf";

        var persona = new Persona();
        persona.setNombre("Juan");
        persona.setApellido("Perez");
        persona.setEmail("jperez@mail.com");
        persona.setTelefono("55443322");

        log.info("ejecutando el controlador Spring MVC");
        model.addAttribute("mensaje", mensaje);
        model.addAttribute("saludo", saludo);
        model.addAttribute("persona", persona);
        return "index";
    }
}
```

Paso 3: Editar la vista index.html

Ubicación:

src/main/resources/templates/index.html

▼ Modifica el contenido del archivo index.html para mostrar los valores de la clase Persona:

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
    <head>
```

```
<title>Inicio</title>
<meta charset="UTF-8">
</head>
<body>
  <h1>Inicio</h1>
  <p th:text="${mensaje}"></p>
  <p th:text="${saludo}"></p>

  <h2>Datos de la Persona</h2>
  <p th:text="'Nombre: ' + ${persona.nombre}">Nombre</p>
  <p th:text="'Apellido: ' + ${persona.apellido}">Apellido</p>
  <p th:text="'Email: ' + ${persona.email}">Email</p>
  <p th:text="'Telefono: ' + ${persona.telefono}">Telefono</p>
</body>
</html>
```

✚ Explicación:

- Usamos `th:text` para mostrar el valor de los atributos que recibimos del backend.
- `${persona.nombre}` accede al atributo `nombre` del objeto `persona`.
- Al refrescar la página en el navegador, verás la información renderizada directamente.



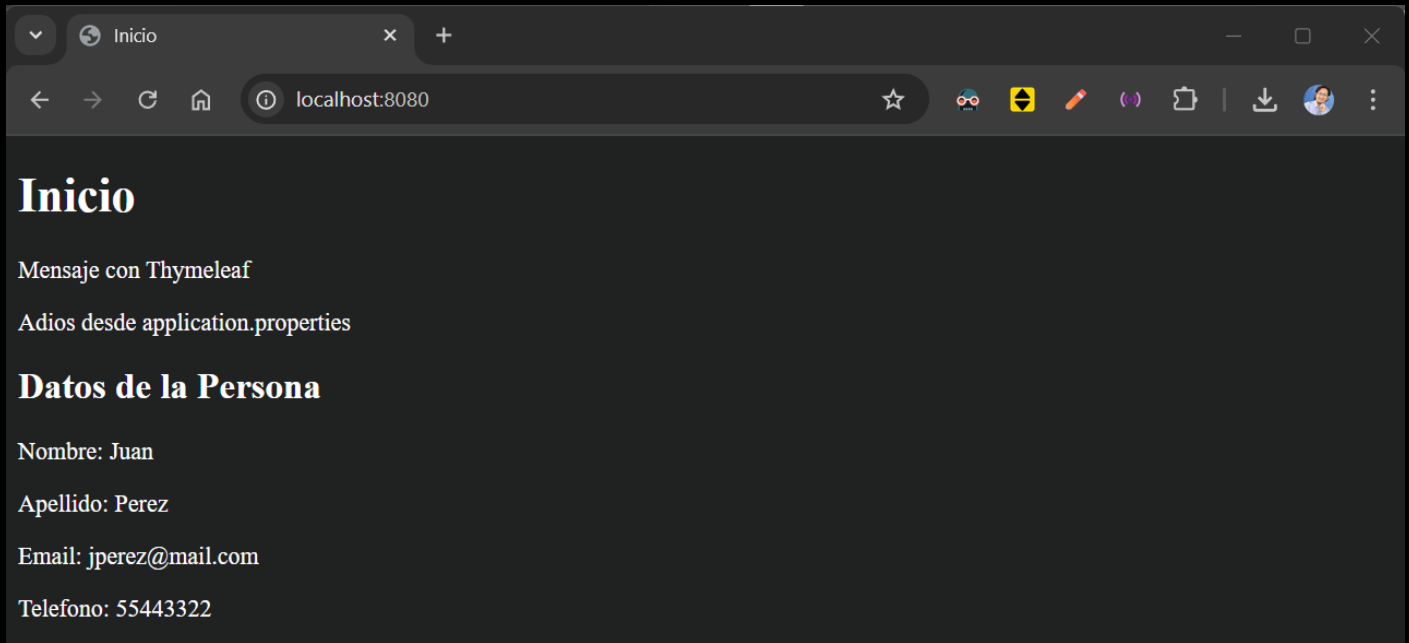
Paso 4. Ejecutar y Verificar

1. Haz clic derecho sobre `HolaSpringApplication.java` > Run File.
2. Abre el navegador y entra a:

<http://localhost:8080/>



Verás ambos mensajes:



Conclusión

Con esta lección completamos el ciclo básico del **patrón MVC en Spring Boot**:

- Creamos un modelo (`Persona`)
- Lo enviamos desde el controlador a la vista
- Mostramos sus datos con **Thymeleaf**

Esto te permitirá crear aplicaciones web dinámicas y organizadas 🎉.

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)